MINI ERP SYSTEM — PRODUCTION ARCHITECTURE CONTROL REPORT

1. Purpose

The purpose of this document is not to describe the project, but to define architectural rules that make the system resilient to failure, difficult to misuse, and operationally reliable.

This report:

- *enforces correct layering of code*
- *prevents invalid dependencies*
- *integrates failure scenarios into the system design*
- *makes architectural violations visible and controllable*
- *enforces development discipline systematically*

2. Design Principle

The system is not designed to work correctly, but to make incorrect behavior difficult.

The Excel + VBA platform has inherent limitations. Therefore, the goal is:

- *not to hide platform limitations*
- *but to accept them*
- *and build a highly disciplined mini ERP core despite them*

3. Architecture

3.1 Layers

- *UI (Forms)*
- *Application (Optional)*
- *Service*
- *Policy*
- *Repository*
- *Infrastructure*

3.2 Dependency Flow

Forms → App → Service → Policy → Repository → Infrastructure

(If Application layer is not used: Forms → Service)

3.3 Forbidden Dependencies

The following relationships are strictly prohibited:

- *Form → Repository*
- *Form → Query*
- *Application → Repository*
- *Repository → Service*
- *Policy → Repository*
- *Query → Write*
- *Entity → Service*

4. Data Integrity (Source of Truth)

The system distinguishes between derived data and actual data.

Actual Data Sources

- *Stock → StockMovements*
- *Balance → CustomerLedger*

Snapshot Fields

- *Product.CurrentStock*
- *Customer.Balance*

Rules

- *cannot be modified by the user*
- *updated only by services*
- *stored for performance purposes*
- *not considered as source of truth*
- *validated through reconciliation*

5. Document Lifecycle

The document lifecycle is fixed:

- *Draft*
- *Posting*
- *Posted*
- *Cancelled*
- *RecoveryRequired*

Critical Distinction

- *Draft → no business impact yet*
- *RecoveryRequired → incomplete/failed transaction*

Without this distinction, system reliability is compromised.

6. Transaction Model

Excel does not provide transactional guarantees. Therefore:

Controlled Staged Write + Final Commit

1. *Create Draft*
2. *Write line items*
3. *Perform validation*
4. *Generate side effects*
5. *Write audit log*
6. *Set status = Posted*
7. *Idempotency*

Rule

A document is considered complete only when it is in **Posted** state. All write operations must be repeatable.

Requirements

- *must include a CorrelationId*
- *must not create duplicates when retried*

Storage Model

ProcessedOperations:

- *CorrelationId*
- *OperationType*
- *EntityId*
- *Status*
- *ProcessedAt*

Rules

- *unique key: CorrelationId + OperationType*
- *duplicate operations must be prevented*
- *retry operations must be safe*

Growth Control

- *retention: 30 days*
- *archiving mechanism*
- *indexed for performance*

8. Side-Effect Tracking

Partial operations must be visible.

Fields

- *StockCreated*
- *LedgerCreated*
- *AuditCreated*

Retry Behavior

- *missing operations are completed*
- *completed operations are not re-executed*

9. Concurrency Model

Locking Strategy

- *document-level locking*

Fields

- *IsLocked*
- *LockedBy*
- *LockedAt*

Rules

- *lock is acquired at the start*
- *lock is released at the end*
- *concurrent access is rejected*

Timeout & Cleanup

- *TTL (e.g., 5 minutes)*
- *expired locks can be cleared*
- *force unlock generates audit logs*

10. Cancel Model

Cancel is not a simple status change.

Definition

Cancel = Business Reversal

Flow

1. *Authorization*
2. *Policy validation*
3. *Reverse stock*
4. *Reverse ledger*
5. *Audit*
6. *Status = Cancelled*

Rule

- *original data is never modified*
- *reversal records are created*

11. Service Layer

The service layer is the core of the system.

Responsibilities

- *orchestration*
- *validation*
- *policy enforcement*
- *repository interaction*
- *idempotency control*
- *concurrency control*
- *side-effect generation*

12. Policy Layer

Policy makes decisions, but does not perform operations.

Examples

- *state transitions*
- *cancel permissions*
- *retry permissions*
- *ownership rules*

13. Query Layer

Query is strictly read-only.

Rules

- read-only
- returns projections
- performs no write operations
- not directly called by UI

14. Read Security

- Normal reads → audit not required
- Sensitive reads → audit required

15. Reconciliation

Ensures system correctness.

Types

- Stock
- Ledger
- Document

Triggers

- after posting
- manual
- dashboard-based

Alarm Quality

- minimize false positives
- group related issues
- merge recurring issues

Critical Rule

If reconciliation does not work, the system is unreliable.

16. Repository Layer

Allowed

- data read/write

Not Allowed

- validation
- authorization
- workflow logic

17. Helper Layer

Allowed

- utility functions

Forbidden

- business logic
- data access

18. Cache Management

- cache ≠ source of truth
- invalidation is mandatory

19. Security & Audit

Security

- enforced at service level
- UI is not considered a security layer

Audit (Mandatory For)

- create
- post
- cancel
- retry
- authentication failure

- force unlock

20. Naming Standards

Type	Prefix
Service	Svc_
Repository	Repo_
Policy	Pol_
Entity	Ent_
Infrastructure	Infra_
Helper	Hlp_
Query	Qry_
Constants	Const_
Form	frm

21. Architectural Violations

Critical

- Form → Repository
- Application → Repository
- Query → Write

High

- validation inside repository
- business logic inside helpers

22. Enforcement

These rules are mandatory.

Controls

- pattern scanning
- regex scanning
- pre-merge validation

Rule

If violations exist → **no merge**

23. Governance

- Critical → blocking
- High → must be fixed
- Exceptions → require written approval

24. Definition of Done

A module is complete if:

- correctly layered
- dependency rules respected
- authorization applied
- audit implemented
- idempotency ensured
- retry-safe

25. Performance

Targets

- form load < 1 second
- dashboard load < 3 seconds

Critical Operations

- posting
- retry
- reconciliation
- lookup

→ must be measurable

26. Minimum Codebase

Initial core:

- Const_*
- Infra_*
- Ent_*
- Repo_*
- Svc_*
- Pol_*
- frm_*

27. Production Risks

- stale locks
- cache inconsistencies
- duplicate operations
- reconciliation failure
- governance breakdown

28. Final Strategy

- do not scale blindly
- harden the system
- enforce the rules

29. Final Verdict

This system is:

- production-grade
- failure-aware
- enforceable
- sustainable